Projeto Final: Sistema de Regressão Espectral com Bagging e Subagging

Reengenharia e Implementação em Streamlit/Python

Wagner Oliveira de Araujo

IFG — 2026

Sumário

1 Levantamento de Requisitos **3**
1.1 Requisitos Funcionais . . . 3
1.2 Entradas e Saídas . . . 4
1.3 Classificação por Prioridade . . . 4
1.4 Requisitos Não Funcionais . . . 5
1.5 Fluxo Principal e Tratamento de Exceções . . . 5

2 Projeto Lógico **5**
2.1 Arquitetura em Camadas . . . 5
2.2 Fluxo de Dados e Lógica de Negócio . . . 6
2.3 Estratégia de Otimização . . . 6

3 Projeto Físico **6**
3.1 Estrutura de Diretórios e Responsabilidades . . . 6
3.2 Ambiente e Dependências . . . 6
3.3 Persistência e Cache . . . 7

4 Implementação **7**
4.1 Destaques da Implementação (`app.py`) . . . 7
4.2 Interface e Fluxo de Execução . . . 7
4.3 Testes, Validação e Rastreabilidade . . . 8
4.4 Comparativo Vaadin → Streamlit . . . 8

1 Levantamento de Requisitos

O levantamento visou catalogar, classificar e priorizar as funcionalidades do sistema legado (Vaadin/Java) para orientar a reengenharia em Python/Streamlit, garantindo rastreabilidade e aderência às necessidades de pesquisadores e alunos de quimiometria.

1.1 Requisitos Funcionais

Tabela 1: Inventário das funções do sistema

Categoria	**Funcionalidade**	**Descrição Técnica**
Cálculos	RLM Simples (Baseline)	Regressão Linear Múltipla padrão sem técnicas de ensemble
Cálculos	RLM+ Bagging	Ensemble com amostragem com reposição (bootstrap)
Cálculos	RLM+ Subagging+ CV	Ensemble sem reposição + otimização por validação cruzada 5-Fold
Configuração	Parâmetro m (Bags)	Define número de amostras/ensembles para Bagging
Configuração	Parâmetro k (Subagging)	Define fração ou quantidade para amostragem no Subagging
Visualização	Métricas de Desempenho	Exibição de R^2, RMSE, MAE no conjunto de teste
Visualização	Gráficos Comparativos	Scatter plots (previsto $\times$ real), resíduos e comparação entre modelos
Organização	Abas de Navegação	Separação lógica: "Arquivos", "Configuração", "Resultados", "Gráficos"
Exportação	Não identificada (legado)	Sistema original sem botões de download; corrigido na reengenharia

1.2 Entradas e Saídas

Tabela 2: Entradas necessárias e saídas esperadas

Tipo	Nome/Conteúdo	Descrição/Formato
	Entradas	
Arquivo	`Xcal.txt`	Matriz de espectros de calibração (variáveis independentes)
Arquivo	`Ycal.txt`	Vetor de valores de referência para calibração
Arquivo	`Xteste.txt`	Matriz de espectros para predição/-teste
Arquivo	`Yteste.txt`	Vetor de valores reais para validação
Parâmetro	m e k	Inteiro ≥ 1 e Decimal $(0, 1]$, respectivamente
	Saídas	
Numérico	R^2, RMSE, MAE	Tabela ou cards destacados
Gráfico	Previsto vs. Real	Scatterplot com reta de ajuste
Gráfico	Resíduos da Predição	Dispersão ou histograma
Comparativo	Desempenho entre modelos	Tabela ou gráfico de barras
Feedback	Mensagens de erro/validação	Alertas em destaque na interface
Status	Indicador de processamento	`st.progress` com etapas descritivas

1.3 Classificação por Prioridade

- **Essenciais (Must Have):** Upload e validação dos 4 arquivos; execução da RLM Simples; cálculo de métricas básicas; validação dimensional (X vs Y); exibição clara dos resultados.
- **Desejáveis (Should Have):** Gráficos interativos (zoom, tooltip); exportação (CSV, TXT, PNG); tooltips explicativos para m e k; pré-processamento básico; histórico de execuções na sessão.
- **Extras (Could Have):** Autenticação e projetos privados; integração com APIs externas; suporte a formatos (.csv, .xlsx, .mat); otimização automática de hiperparâmetros; dashboard comparativo multi-rodada.

1.4 Requisitos Não Funcionais

Tabela 3: Especificação de requisitos não funcionais

Categoria	Especificação	Critério de Aceite
Performance	Tempo de cálculo (RLM/Ensemble)	RLM $< 10\,\text{s}$; Ensemble $< 2\,\text{min}$ (CV 5-Fold)
Performance	Renderização de gráficos	$< 3\,\text{s}$ após conclusão do cálculo
Usabilidade	Acessibilidade (WCAG 2.1 AA)	Contraste, labels, navegação por teclado
Usabilidade	Clareza textual e recuperação de erro	Mensagens orientam correção, não apenas apontam falha
Segurança	Isolamento de sessão e sanitização	Dados não vazam; validação de extensão/conteúdo
Escalabilidade	Crescimento de usuários/dados	Container com load balancer; processamento em batch

1.5 Fluxo Principal e Tratamento de Exceções

O fluxo principal segue: `Upload` → `Validação dimensional` → `Configuração de` m `e` k → `Execução com CV 5-Fold` → `Exibição de resultados e gráficos`. Casos de exceção (arquivo inválido, parâmetros fora do intervalo, timeout) são tratados com validação *fail-fast*, sliders com limites explícitos e mensagens orientativas, garantindo usabilidade e robustez.

2 Projeto Lógico

A reengenharia adotou o modelo clássico de desenvolvimento, promovendo a separação de responsabilidades e a eliminação de acoplamentos presentes no código legado.

2.1 Arquitetura em Camadas

Tabela 4: Diagnóstico do código legado e arquitetura proposta

Aspecto	Estado Atual (Legado)	Solução na Reengenharia
Arquitetura	Monolítica (UI + lógica + parsing)	Camadas separadas: Apresentação, Negócio, Dados
Validação	`try/except` genérico	Função `validate_inputs()` com regras explícitas
Performance	Laços `for` em CV, sem cache	`@st.cache_data`, vetorização NumPy, seed configurável
Tipagem/Doc	Ausente em funções críticas	Type hints + docstrings padrão NumPy
Reprodutibilidade	`seed=42` hardcoded	Parâmetro configurável + registro em logs

2.2 Fluxo de Dados e Lógica de Negócio

1. **Camada de Dados:** Parsing de arquivos `.txt` em `numpy.ndarray`, validação dimensional entre X e Y, e sanitização de conteúdo.

2. **Camada de Negócio:** Implementação vetorizada da RLM ($\hat{\beta} = X^{+}Y$), Bagging/Subagging com reamostragem controlada, e validação cruzada 5-Fold para busca de hiperparâmetros ótimos (m^*, k^*).

3. **Camada de Apresentação:** Interface Streamlit com abas, `st.session_state` para persistência de resultados, e componentes Plotly para visualização interativa.

2.3 Estratégia de Otimização

A validação cruzada 5-Fold particiona aleatoriamente o conjunto de calibração. Para cada par (m, k), calcula-se o RMSE médio e desvio padrão nos folds. O par com menor RMSE-CV é selecionado e aplicado ao conjunto de teste independente, garantindo avaliação não enviesada.

3 Projeto Físico

3.1 Estrutura de Diretórios e Responsabilidades

Tabela 5: Estrutura modular e responsabilidades

Módulo/Arquivo	Função Específica
`app.py`	Camada de apresentação (apenas UI + roteamento de abas)
`src/data_loader.py`	Parsing de `.txt`, validação dimensional, dataclasses
`src/models.py`	Implementação de RLM, Bagging e Subagging (vetorizado)
`src/metrics.py`	Cálculo de RMSE, MAE e R^2 com tipos explícitos
`src/optimizers.py`	Validação cruzada 5-Fold e busca de hiperparâmetros
`tests/`	Suíte de testes unitários (`pytest`) para cada módulo

3.2 Ambiente e Dependências

- **Linguagem/ Framework:** Python 3.10+ e Streamlit 1.30+
- **Cálculo Numérico:** `numpy`, `pandas`
- **Visualização:** `plotly` (gráficos interativos)
- **Testes:** `pytest`, `pytest-cov`
- **Deploy:** Streamlit Community Cloud (gratuito) ou container Docker com `nginx` + `gunicorn` para escalabilidade.

3.3 Persistência e Cache

Utiliza-se `st.session_state` para armazenar resultados entre interações, evitando reprocessamento. A renderização de gráficos é desacoplada do cálculo, e a validação dimensional ocorre antes da alocação em memória, prevenindo `MemoryError` em datasets espectrais grandes.

4 Implementação

A implementação materializa o projeto lógico e físico em código Python, mantendo fidelidade matemática ao original e ampliando usabilidade e reprodutibilidade.

4.1 Destaques da Implementação (`app.py`)

```python
def perform_rlm(Xcal, Ycal, Xteste):
    """RLM simples via pseudoinversa."""
    beta = np.linalg.pinv(Xcal) @ Ycal
    return Xteste @ beta

def perform_bagging(Xcal, Ycal, Xteste, num_bags, with_replacement, fraction, seed=42):
    """RLM + Bagging / Subagging com vetorização e seed configurável."""
    n = Xcal.shape[0]
    sample_size = n if with_replacement else max(1, int(n * fraction))
    rng = np.random.default_rng(seed)
    Ypred = np.zeros(Xteste.shape[0])
    for _ in range(num_bags):
        idx = rng.integers(0, n, size=sample_size) if with_replacement else \
              rng.choice(n, size=sample_size, replace=False)
        beta = np.linalg.pinv(Xcal[idx]) @ Ycal[idx]
        Ypred += (Xteste @ beta) / num_bags
    return Ypred
```

4.2 Interface e Fluxo de Execução

- **Upload:** 4 arquivos `.txt` com validação imediata de formato e dimensões.
- **Configuração:** Inputs para m e k com parsing robusto e tooltips.
- **Execução:** `st.progress` com *callback* atualizado a cada iteração de CV.
- **Resultados:** Cards estilizados, tabela `pandas` exportável para CSV, e log detalhado.
- **Gráficos:** Plotly com `Scatter`, linha de referência $y = x$, e tooltips interativos.

4.3 Testes, Validação e Rastreabilidade

Tabela 6: Estratégia de testes e critérios de aceite

Tipo de Teste	**Escopo**	**Meta de Cobertura**
Unitário	Funções isoladas (parse, metrics)	$\geq 80\%$ (`pytest-cov`)
Integração	Fluxo completo: Upload $\rightarrow$ CV $\rightarrow$ Gráficos	100% dos caminhos críticos
Performance	Tempo de resposta para datasets típicos	RLM $< 10\,\text{s}$; Ensemble $< 2\,\text{min}$
Aceite (UAT)	Validação por pesquisadores/alunos	SUS ≥ 75 (Bom/Excelente)

Tabela 7: Matriz de rastreabilidade de requisitos (RTM)

ID Requisito	**Fase Clássica**	**Artefato de Validação**
RF01.1–01.5	1, 3, 5	`data_loader.py`, testes de upload
RF02.1–02.3	1, 2, 4	`optimizers.py`, logs de execução
RF03.1–03.3	2, 3, 4	`models.py`, testes unitários de métricas
RF04.1–04.3	2, 4, 5	`metrics.py`, UAT visual (Plotly)
RF05.1–05.3	1, 3, 5	UI/UX, feedback de erros, `st.progress`

4.4 Comparativo Vaadin $\rightarrow$ Streamlit

- `@Route()` $\rightarrow$ `st.set_page_config()`
- `Tabs` $\rightarrow$ `st.tabs()`
- `Upload + MemoryBuffer` $\rightarrow$ `st.file_uploader()`
- `CompletableFuture` $\rightarrow$ Execução síncrona com `st.progress`
- `RegressionService (Java)` $\rightarrow$ Funções Python com NumPy vetorizado
- `EJML SimpleMatrix` $\rightarrow$ `numpy.linalg.pinv()`

A implementação resultante atende aos requisitos funcionais e não funcionais levantados, reduz a curva de aprendizado para usuários não especialistas, e estabelece base técnica sólida para evoluções futuras (autenticação, pipelines automatizados, suporte a novos formatos).